

Phase 2 Frontend Implementation Guide

1. What This Document Is For

This is the ****detailed Phase 2 frontend implementation guide**** for the Aivan HRMS Portal.

It is written so a developer can understand:

- what Phase 2 should include on the frontend
- which new screens should be built
- which current screens should be updated
- how routing should expand
- what services and models are required
- how frontend should consume backend APIs

This document assumes:

- Phase 1 corrections are being completed first
- the current Angular app structure remains the base
- backend APIs will become the source of truth

2. Current Frontend Baseline

The current Angular frontend already contains:

- login
- forgot password
- reset password
- admin dashboard
- HR dashboard
- employee dashboard
- employee management
- attendance screens
- profile
- change password
- login activity
- navbar notifications

This means Phase 2 should be built as ****extension and cleanup****, not as a restart.

Relevant existing files:

- [app.routes.ts](/Users/vivekmehta/Development/Vivek/AIVan/Aivan-HRMS-Portal/frontend/src/app/app.routes.ts)
- [attendance.service.ts](/Users/vivekmehta/Development/Vivek/AIVan/Aivan-HRMS-Portal/frontend/src/app/services/attendance.service.ts)
- [notification.service.ts](/Users/vivekmehta/Development/Vivek/AIVan/Aivan-HRMS-Portal/frontend/src/app/services/notification.service.ts)
- [emp-dashboard.ts](/Users/vivekmehta/Development/Vivek/AIVan/Aivan-HRMS-Portal/frontend/src/app/features/emp-dashboard.ts)
- [hr-dashboard.ts](/Users/vivekmehta/Development/Vivek/AIVan/Aivan-HRMS-Portal/frontend/src/app/features/hr-dashboard.ts)

3. Phase 2 Frontend Goals

Phase 2 frontend should achieve 5 things:

1. complete the partially visible operational modules already hinted in the UI
2. move page behavior from demo-style to API-driven behavior
3. make approval workflows easy to use
4. make admin and HR operations more structured
5. keep role experience clean inside the same shared portal

4. Phase 2 Frontend Scope

New Functional Areas

1. 'Time Off Module'
2. 'Approval Center'
3. 'Attendance Regularization'
4. 'Master Data Management'
5. 'Reports and Exports'
6. 'Security and Login Activity Improvements'

Existing Areas To Update

1. employee dashboard
2. HR dashboard
3. admin dashboard
4. employee attendance page
5. navbar notification handling
6. employee and HR list pages

5. Recommended Frontend Folder Expansion

Keep the current Phase 1 structure, and add these areas:

```
'''text
frontend/src/app/
??? core/
?   ??? models/
?   ?   ??? timeoff.model.ts
?   ?   ??? approval.model.ts
?   ?   ??? regularization.model.ts
?   ?   ??? master-data.model.ts
?   ?   ??? report.model.ts
?   ??? services/
?       ??? timeoff.service.ts
?       ??? approval.service.ts
?       ??? regularization.service.ts
?       ??? master-data.service.ts
?       ??? report.service.ts
??? features/
?   ??? admin/
?   ?   ??? pages/
?   ?       ??? master-data/
?   ?       ??? holiday-calendar/
?   ?       ??? reports/
?   ??? hr/
?   ?   ??? pages/
?   ?       ??? approval-center/
?   ?       ??? regularization-requests/
?   ?       ??? leave-requests/
?   ?       ??? reports/
?   ??? emp/
?   ?   ??? pages/
?   ?       ??? time-off/
?   ?       ??? regularization/
?   ?       ??? notifications/
?   ??? shared-workflows/
?       ??? approval-status-chip/
?       ??? request-timeline/
?       ??? export-toolbar/
```

'''

6. Frontend Module Plan

6.1 Time Off Module

Employee Screens

1. 'Time Off Request Page'
 - request form
 - leave type dropdown
 - date picker
 - half-day or hourly controls
 - reason textarea
 - leave balance summary
 - holidays or blackout warnings
2. 'My Time Off Requests'
 - pending, approved, rejected tabs
 - request history list
 - request detail drawer or modal
 - cancel option only for allowed states

HR/Admin Screens

1. 'Leave Requests List'
 - pending filter by default
 - employee name
 - date range
 - leave type
 - reason preview
 - action buttons
2. 'Leave Request Decision Modal'
 - approve
 - reject
 - approval comment
 - approved duration override if allowed

Existing Screen Update

The employee dashboard currently shows a time off block.

In Phase 2, decide one of these approaches and apply consistently:

- keep dashboard quick-apply and link to full page
- or simplify dashboard and move full workflow into dedicated page

Recommended approach:

- dashboard should keep a small quick-entry widget
- full request history and advanced controls should move to '/emp-dashboard/time-off'

6.2 Approval Center

HR Approval Center Screen

This should become the main operations page for HR.

Sections:

- pending time off approvals
- pending attendance regularization approvals
- recently processed approvals
- approval counters by type

Actions:

- approve
- reject
- view request details
- view employee context

Admin Approval Oversight Screen

Admin does not need to process every request by default, but should be able to:

- monitor pending approval volume
- view decision logs
- step in if HR escalation is needed

6.3 Attendance Regularization

Employee Screens

1. 'Request Attendance Correction'
 - select date
 - choose reason type
 - enter expected punch-in
 - enter expected punch-out
 - enter explanation
 - upload proof later if Phase 3 or above, not required now
2. 'My Regularization Requests'
 - list by status
 - show submitted dates
 - show HR comments

HR Screens

1. 'Regularization Requests'
 - employee
 - requested date
 - current recorded values
 - requested corrected values
 - reason
 - approve/reject action

Existing Screen Update

Update employee attendance page so the user can raise a correction request directly from a problematic row.

6.4 Master Data Management

Admin Pages

1. 'Departments'

2. 'Designations'
3. 'Shifts'
4. 'Work Locations'
5. 'Leave Types'
6. 'Holiday Calendar'

Each page should support:

- list
- search
- add
- edit
- activate/deactivate

Why This Matters

Right now many forms depend on dropdown values that are either hardcoded or loosely controlled.

Phase 2 should make these values admin-managed and backend-driven.

6.5 Reports and Exports

HR Reports

1. attendance summary report
2. employee late arrival report
3. missing punch report
4. leave usage report

Admin Reports

1. HR workload report
2. employee status summary
3. login activity summary

UI Requirements

- filter panel
- export buttons
- empty-state handling
- table pagination
- date range support

6.6 Security and Login Activity Improvements

The login activity area already exists.

Phase 2 should make it more complete:

- add stronger filters
- show status badges clearly
- support role-based visibility rules
- connect notifications to login activity records reliably

Also align:

- forgot password
- reset password

- change password

so they look like one connected user security flow, not separate isolated pages.

7. Recommended Phase 2 Route Map

Suggested route additions:

Path	Role	Access	Purpose
--- --- ---			
'/emp-dashboard/time-off'	employee, admin		employee time off page
'/emp-dashboard/regularization'	employee, admin		employee attendance correction page
'/hr-dashboard/approval-center'	hr, admin		all pending approvals
'/hr-dashboard/leave-requests'	hr, admin		leave request management
'/hr-dashboard/regularization-requests'	hr, admin		attendance correction approvals
'/master-dashboard/master-data/departments'	admin		manage departments
'/master-dashboard/master-data/designations'	admin		manage designations
'/master-dashboard/master-data/shifts'	admin		manage shifts
'/master-dashboard/master-data/locations'	admin		manage locations
'/master-dashboard/master-data/leave-types'	admin		manage leave types
'/master-dashboard/master-data/holidays'	admin		manage holidays
'/hr-dashboard/reports'	hr, admin		HR operational reports
'/master-dashboard/reports'	admin		admin reports

8. Frontend Service Layer Plan

Create or refactor these services:

'timeoff.service.ts'

Responsibilities:

- fetch leave balance
- fetch leave request list
- submit request
- cancel request
- fetch HR leave approval list
- approve or reject request

'approval.service.ts'

Responsibilities:

- fetch all pending approvals
- fetch processed approvals
- approval counts by type
- unified approval actions

'regularization.service.ts'

Responsibilities:

- submit attendance correction
- fetch own requests
- fetch HR review queue
- approve or reject request

'master-data.service.ts'

Responsibilities:

- load dropdown bootstrap data
- CRUD for departments, designations, shifts, locations, leave types, holidays

'report.service.ts'

Responsibilities:

- fetch report tables
- export CSV links or blobs

9. Frontend Model Additions

Suggested models:

'timeoff.model.ts'

- 'TimeOffRequest'
- 'TimeOffBalance'
- 'TimeOffDecisionPayload'
- 'TimeOffBootstrap'

'approval.model.ts'

- 'ApprovalQueueItem'
- 'ApprovalDecisionPayload'
- 'ApprovalSummary'

'regularization.model.ts'

- 'AttendanceRegularizationRequest'
- 'AttendanceRegularizationDecision'
- 'AttendanceRegularizationListItem'

'master-data.model.ts'

- 'Department'
- 'Designation'
- 'Shift'
- 'WorkLocation'
- 'LeaveType'
- 'Holiday'

'report.model.ts'

- 'AttendanceReportRow'
- 'LeaveReportRow'
- 'ExceptionReportRow'

9.1 Exact Frontend Type Definitions

Below is the recommended detail level for frontend model files.

These are not random examples.

These should be treated as the working contract for Angular development.

'timeoff.model.ts'

```
'''ts
export type RequestStatus = 'pending' | 'approved' | 'rejected' | 'cancelled';
export type DurationType = 'full_day' | 'half_day' | 'hourly';

export interface TimeOffLeaveType {
  id: number;
  name: string;
  code: string;
  unitType: DurationType;
  requiresApproval: boolean;
  defaultBalanceHours: number;
  isActive: boolean;
}

export interface TimeOffBalance {
  totalHours: number;
  usedHours: number;
  remainingHours: number;
}

export interface HolidayItem {
  id: number;
  date: string;
  name: string;
  description?: string | null;
  isOptional: boolean;
}

export interface TimeOffBootstrap {
  leaveTypes: TimeOffLeaveType[];
  balance: TimeOffBalance;
  holidays: HolidayItem[];
}

export interface TimeOffCreatePayload {
  leaveTypeId: number;
  dateFrom: string;
  dateTo: string;
  durationType: DurationType;
  startTime?: string | null;
  endTime?: string | null;
  reason: string;
}

export interface TimeOffRequestItem {
  id: number;
  employeeId: number;
  employeeName?: string;
  leaveTypeId: number;
  leaveTypeName: string;
  dateFrom: string;
  dateTo: string;
  durationType: DurationType;
  requestedHours: number;
  approvedHours?: number | null;
  reason: string;
  status: RequestStatus;
  reviewComment?: string | null;
  createdAt: string;
  reviewedAt?: string | null;
}
```



```

export interface TimeOffDecisionPayload {
  decision: 'approved' | 'rejected';
  comment: string;
  approvedHours?: number | null;
}
'''

'approval.model.ts'

'''ts
export type ApprovalRequestType = 'timeoff' | 'regularization';
export type ApprovalPriority = 'low' | 'normal' | 'high';

export interface ApprovalQueueItem {
  id: number;
  requestType: ApprovalRequestType;
  requestId: number;
  employeeId: number;
  employeeName: string;
  submittedAt: string;
  submittedBy: number;
  status: 'pending' | 'approved' | 'rejected';
  priority: ApprovalPriority;
  summaryText: string;
}

export interface ApprovalCounts {
  timeoff: number;
  regularization: number;
  total: number;
}

export interface ApprovalQueueResponse {
  items: ApprovalQueueItem[];
  counts: ApprovalCounts;
}
'''

'regularization.model.ts'

'''ts
export type RegularizationReasonType =
  | 'missed_punch'
  | 'late_sync'
  | 'forgot_punch_out'
  | 'forgot_punch_in'
  | 'system_issue'
  | 'other';

export interface RegularizationCreatePayload {
  attendanceDate: string;
  requestedPunchIn?: string | null;
  requestedPunchOut?: string | null;
  reasonType: RegularizationReasonType;
  reasonText: string;
}

export interface RegularizationRequestItem {
  id: number;
  employeeId: number;
  employeeName?: string;
  attendanceDate: string;
  currentPunchIn?: string | null;

```

```
currentPunchOut?: string | null;
requestedPunchIn?: string | null;
requestedPunchOut?: string | null;
reasonType: RegularizationReasonType;
reasonText: string;
status: 'pending' | 'approved' | 'rejected';
reviewComment?: string | null;
createdAt: string;
reviewedAt?: string | null;
}
'''
```

```
'master-data.model.ts'
```

```
'''ts
```

```
export interface Department {
  id: number;
  name: string;
  code: string;
  description?: string | null;
  isActive: boolean;
}
```

```
export interface Designation {
  id: number;
  name: string;
  code: string;
  description?: string | null;
  isActive: boolean;
}
```

```
export interface Shift {
  id: number;
  name: string;
  code: string;
  startTime: string;
  endTime: string;
  weeklyOffDays: string[];
  graceInMinutes: number;
  graceOutMinutes: number;
  isActive: boolean;
}
```

```
export interface WorkLocation {
  id: number;
  name: string;
  code: string;
  address?: string | null;
  isActive: boolean;
}
```

```
export interface LeaveType {
  id: number;
  name: string;
  code: string;
  unitType: 'full_day' | 'half_day' | 'hourly';
  defaultBalanceHours: number;
  requiresApproval: boolean;
  isActive: boolean;
}
```

```
export interface Holiday {
  id: number;
```

```

    holidayDate: string;
    name: string;
    description?: string | null;
    isOptional: boolean;
    isActive: boolean;
}
'''

'report.model.ts'

'''ts
export interface AttendanceReportRow {
    employeeId: number;
    employeeCode: string;
    employeeName: string;
    departmentName: string;
    attendanceDate: string;
    punchIn?: string | null;
    punchOut?: string | null;
    workingHours: number;
    attendanceStatus: string;
}

export interface LeaveReportRow {
    employeeId: number;
    employeeName: string;
    leaveTypeName: string;
    requestedHours: number;
    approvedHours: number;
    status: string;
    requestDate: string;
}

export interface ExceptionReportRow {
    employeeId: number;
    employeeName: string;
    exceptionType: string;
    attendanceDate: string;
    remarks: string;
}
'''

---
```

9.2 Shared Response Wrappers

Use common wrappers so the fresher developer knows what every API result should look like.

```

'''ts
export interface ApiSuccessResponse<T> {
    success: true;
    message: string;
    data: T;
}

export interface PaginatedResponse<T> {
    items: T[];
    page: number;
    pageSize: number;
    totalItems: number;
    totalPages: number;
}
'''
```

```
export interface ApiErrorResponse {
  success: false;
  message: string;
  errors?: Record<string, string[]>;
}
'''
```

9.3 Which Fields Are Required And Optional

Time Off Create Form

- 'leaveTypeId': required 'number'
- 'dateFrom': required 'string' in 'YYYY-MM-DD'
- 'dateTo': required 'string' in 'YYYY-MM-DD'
- 'durationType': required 'string'
- 'startTime': optional for full day, required for hourly
- 'endTime': optional for full day, required for hourly
- 'reason': required 'string', minimum 5 characters

Regularization Create Form

- 'attendanceDate': required 'string'
- 'requestedPunchIn': optional 'string | null'
- 'requestedPunchOut': optional 'string | null'
- 'reasonType': required 'string'
- 'reasonText': required 'string', minimum 10 characters

Master Data Form

- 'name': required 'string'
- 'code': required 'string'
- 'description': optional 'string | null'
- 'isActive': required 'boolean'

9.4 Validation Rules To Implement In Angular

Use Angular reactive forms and explicitly apply these validations:

Time Off Form

- 'leaveTypeId': 'Validators.required'
- 'dateFrom': 'Validators.required'
- 'dateTo': 'Validators.required'
- 'reason': 'Validators.required', 'Validators.minLength(5)', 'Validators.maxLength(500)'
- 'startTime' and 'endTime': required only when 'durationType === 'hourly''

Regularization Form

- 'attendanceDate': 'Validators.required'
- 'reasonType': 'Validators.required'
- 'reasonText': 'Validators.required', 'Validators.minLength(10)', 'Validators.maxLength(1000)'

Master Data Forms

- 'name': 'Validators.required', 'Validators.maxLength(100)'
- 'code': 'Validators.required', 'Validators.maxLength(30)'
- 'description': 'Validators.maxLength(250)'

9.5 Suggested Angular File Map Per Feature

Time Off

```
'''text
features/emp/pages/time-off/
??? time-off.ts
??? time-off.html
??? time-off.css
??? components/
?   ??? time-off-form/
?   ??? time-off-history-table/
?   ??? time-off-balance-card/
'''
```

Approval Center

```
'''text
features/hr/pages/approval-center/
??? approval-center.ts
??? approval-center.html
??? approval-center.css
??? components/
?   ??? approval-summary-cards/
?   ??? approval-table/
?   ??? approval-decision-modal/
'''
```

Regularization

```
'''text
features/emp/pages/regularization/
??? regularization.ts
??? regularization.html
??? regularization.css
??? components/
?   ??? regularization-form/
?   ??? regularization-history/
'''
```

9.6 Frontend Service Method Signatures

Use method signatures like this so implementation stays predictable:

```
'''ts
getTimeOffBootstrap(): Observable<ApiSuccessResponse<TimeOffBootstrap>>;
createTimeOff(payload: TimeOffCreatePayload):
Observable<ApiSuccessResponse<TimeOffRequestItem>>;
getMyTimeOffRequests(page: number, pageSize: number, status?: string):
Observable<PaginatedResponse<TimeOffRequestItem>>;
decideTimeOff(requestId: number, payload: TimeOffDecisionPayload):
Observable<ApiSuccessResponse<TimeOffRequestItem>>;

createRegularization(payload: RegularizationCreatePayload):
Observable<ApiSuccessResponse<RegularizationRequestItem>>;
getMyRegularizations(page: number, pageSize: number, status?: string):
Observable<PaginatedResponse<RegularizationRequestItem>>;
decideRegularization(id: number, payload: { decision: 'approved' | 'rejected'; comment: string
}): Observable<ApiSuccessResponse<RegularizationRequestItem>>;
```

```
getApprovalQueue(): Observable<ApiSuccessResponse<ApprovalQueueResponse>>;
getDepartments(page: number, pageSize: number, search?: string):
Observable<PaginatedResponse<Department>>;
'''
```

10. UI/UX Rules For Phase 2

Keep Existing UI Style

The current portal already has a visual style.

Phase 2 should:

- reuse current navbar and sidebar style
- reuse current cards and table style
- reuse badge and form patterns
- avoid introducing a totally different design language

Improve Consistency

Areas to clean up:

- button labels
- form spacing
- modal width and structure
- table action placement
- status badge colors
- error message handling

Status Colors

Use the same status color pattern across attendance, leave, and approval screens:

- pending
- approved
- rejected
- active
- inactive

11. API Integration Rules

Frontend should stop doing backend work where possible.

That means:

- filtering should move to backend
- pagination should move to backend
- list summary counts should come from backend
- calculation-heavy dashboards should use backend aggregates

Frontend should still keep:

- local loading states
- form validation
- UI-only formatting
- optimistic status handling where safe

12. Step-By-Step Frontend Build Order

Follow this order:

Step 1

Clean and align existing routes and shared navigation.

Step 2

Create service files and models for:

- time off
- approvals
- regularization
- master data
- reports

Step 3

Build employee time off page and connect it to backend.

Step 4

Build HR approval center and leave request review screens.

Step 5

Build attendance regularization request and approval flow.

Step 6

Build admin master data pages.

Step 7

Build reports and export screens.

Step 8

Refine notifications, login activity linking, and empty states.

13. Frontend Definition Of Done For Phase 2

Phase 2 frontend is done when:

- all new pages are routed and accessible by correct roles
- all forms submit to real APIs
- pending and processed requests display correctly
- approval actions update UI without page confusion
- master data pages are CRUD-capable
- reports load with filters and export action
- notifications and login activity feel connected
- no important page depends on hardcoded dropdown data

14. Notes For The Developer

If one developer is doing both frontend and backend:

- build models and service contracts first
- do not start styling every page before API shape is clear
- finish one workflow fully before opening the next one

Recommended workflow:

1. time off full loop
2. approval center
3. regularization
4. master data
5. reports

That order will keep the work understandable and easier to test.

15. Fresher Developer Notes

If the developer is very new, tell him to work in this exact frontend order:

1. read existing route file
2. create model interfaces first
3. create service methods next
4. create form page after service contract is fixed
5. bind form submission to backend
6. then create history table
7. then create filters and pagination

Important rule:

Do not directly start with HTML design first.

First define:

- what object is coming from backend
- what object is going to backend
- what fields are required
- what screen state is needed

Then build the component.